

Islamic University of Technology

Department of Software Engineering

Software Project Lab II (SWE 4404)

Project title

AmarShohor

(A Community-Driven Urban Issue Mapping Platform)

Software Design Report

Supervisor

Yousuf Ibne kamal Niloy

Group: 02

Team Members

Procheta Silvie (230042114)

Tasnia Farzana (230042124)

Aksan Anan Ria (230042154)

Contents

1 Introduction **3**

1.1 Main Objectives 3

1.2 Previous Detailed SRS 3

2 Software Design Approach **4**

3 UML Class Diagram **5**

3.1 Brief Explanation 5

4 Three-Tier Software Architecture **6**

4.1 Brief Explanation 7

5 Software Requirements Specification (SRS) **7**

5.1 Functional Requirements 7

5.2 Non-Functional Requirements 23

6 Conclusion **24**

1. Introduction

AmarShohor is a community-driven urban issue mapping platform designed for cities in Bangladesh. It provides citizens with a structured and location-aware way to report common urban problems such as damaged roads, garbage accumulation, waterlogging, electrical faults, unsafe areas, and emergency incidents. Citizens can create reports by selecting an issue category, providing a description, adding a geographical location, and uploading photographs, videos, or other supporting evidence. Reported issues are displayed in a community feed where other users can confirm, discuss, or flag them. The platform also uses map and heatmap visualizations to help users understand the geographical distribution and concentration of urban problems. AmarShohor also supports organized volunteer participation. Citizens may apply to become volunteers by providing their skills, preferred service areas, and availability. Suitable volunteers can view nearby tasks, accept assignments, update progress, and submit evidence after completing work. Citizens can then provide feedback and ratings regarding the completed task. The platform does not depend on a conventional internal administrator for the main validation process. Community confirmation, automated rules, evidence verification, and transparent status tracking are used to reduce bias and promote accountability. External stakeholders, such as emergency services, map providers, and notification gateways, may interact with the system when necessary.

1.1. Main Objectives

The main objectives of AmarShohor are:

- To provide a centralized platform for reporting urban issues.
- To display issues through location-based maps and heatmaps.
- To support transparent community validation and discussion.
- To prevent unnecessary duplicate reports.
- To organize volunteer participation and task management.
- To support emergency reporting and priority handling.
- To provide notifications, escalation, feedback, and public analytics.

1.2. Previous Detailed SRS

The complete Software Requirements Specification, including functional requirements, user stories, acceptance criteria, priorities, and current implementation status, is available through the following Google Sheet:

AmarShohor Detailed Software Requirements Specification Google Sheet

2. Software Design Approach

AmarShohor follows an Object-Oriented software design approach. The system is represented using domain objects such as `User`, `Citizen`, `Volunteer`, `Report`, `Task`, `Evidence`, `Comment`, `Notification`, `Feedback`, `Category`, and `Location`. Each object contains related attributes and operations, which makes the system easier to understand, develop, test, and maintain. The design applies the following principles:

- **Encapsulation:** Related data and behavior are placed inside the appropriate class.
- **Inheritance:** The `Citizen` and `Volunteer` classes inherit common account information and operations from the `User` class.
- **Single Responsibility:** Each class, service, and repository is responsible for a specific part of the system.
- **Separation of Concerns:** Domain models, business rules, database operations, and stored data remain separated.
- **Modularity:** Major features, such as reporting, community validation, volunteer management, task handling, and notifications, are organized into independent modules.
- **Extensibility:** New report categories, notification types, rewards, or analytics features can be added without redesigning the entire system.

Along with Object-Oriented design, AmarShohor follows a feature-wise three-tier layered architecture. Each major feature is organized through the following flow:

Model ⟶ Business Logic ⟶ Data Access Layer ⟶ PostgreSQL Database

3. UML Class Diagram

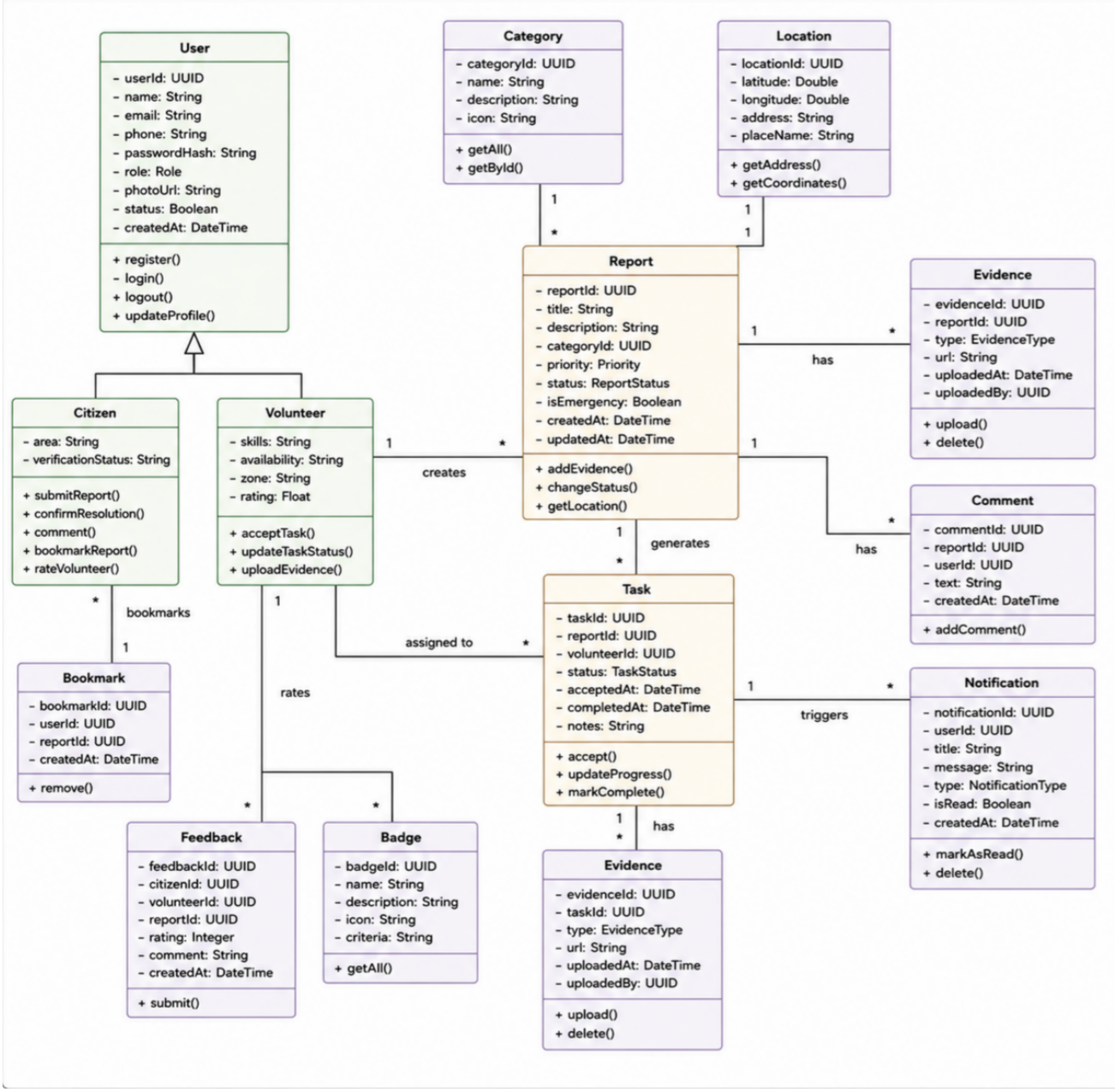


Figure 1: UML Class Diagram of AmarShohor

3.1. Brief Explanation

Figure 1 illustrates the major classes and relationships of the AmarShohor system. The `User` class stores common account information such as user ID, name, email, phone number, password hash, role, profile photograph, account status, and creation date. It also contains common operations such as registration, login, logout, and profile updating. The `Citizen` and `Volunteer` classes inherit from `User`. A citizen can submit reports, confirm resolutions, comment on reports, bookmark important reports, and rate volunteer work. A volunteer can accept tasks, update task status, and upload completion evidence. The `Report` class represents an urban issue and stores information such as title, description, category,

priority, status, emergency condition, and timestamps. Each report is connected to a `Category` and a `Location`. A report may contain multiple pieces of `Evidence` and multiple `Comment` objects. A validated report may generate a `Task`. The task is assigned to a volunteer and stores the task status, acceptance time, completion time, notes, and completion evidence. The `Notification` class informs users about report and task-related changes. The `Feedback` class stores ratings and comments submitted after task completion. The `Badge` class represents achievement rewards earned by volunteers, while the `Bookmark` class stores reports saved by citizens for later viewing.

4. Three-Tier Software Architecture

AmarShohor uses a feature-wise three-tier architecture. Because the platform contains several major modules, the architecture is divided into two diagrams to keep the design readable.

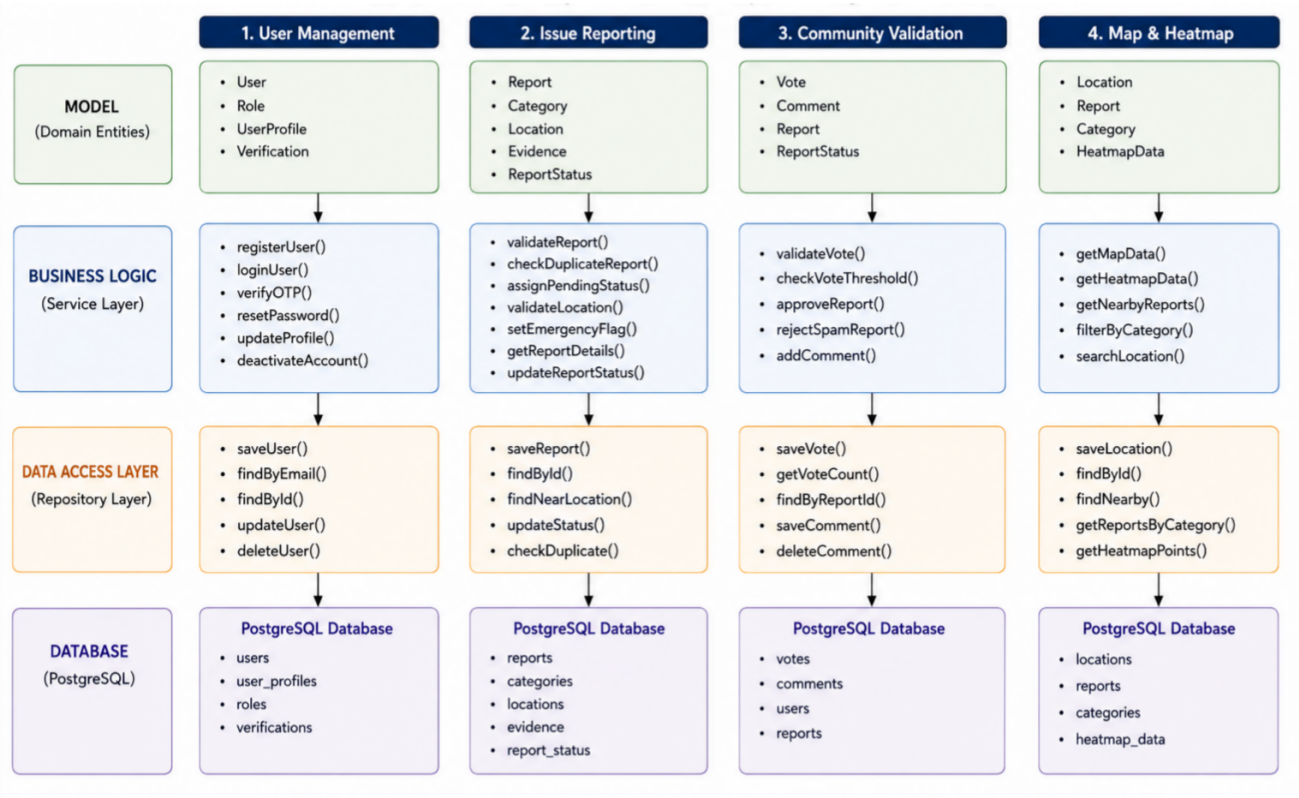


Figure 2: Feature-Wise Three-Tier Architecture – Diagram 1

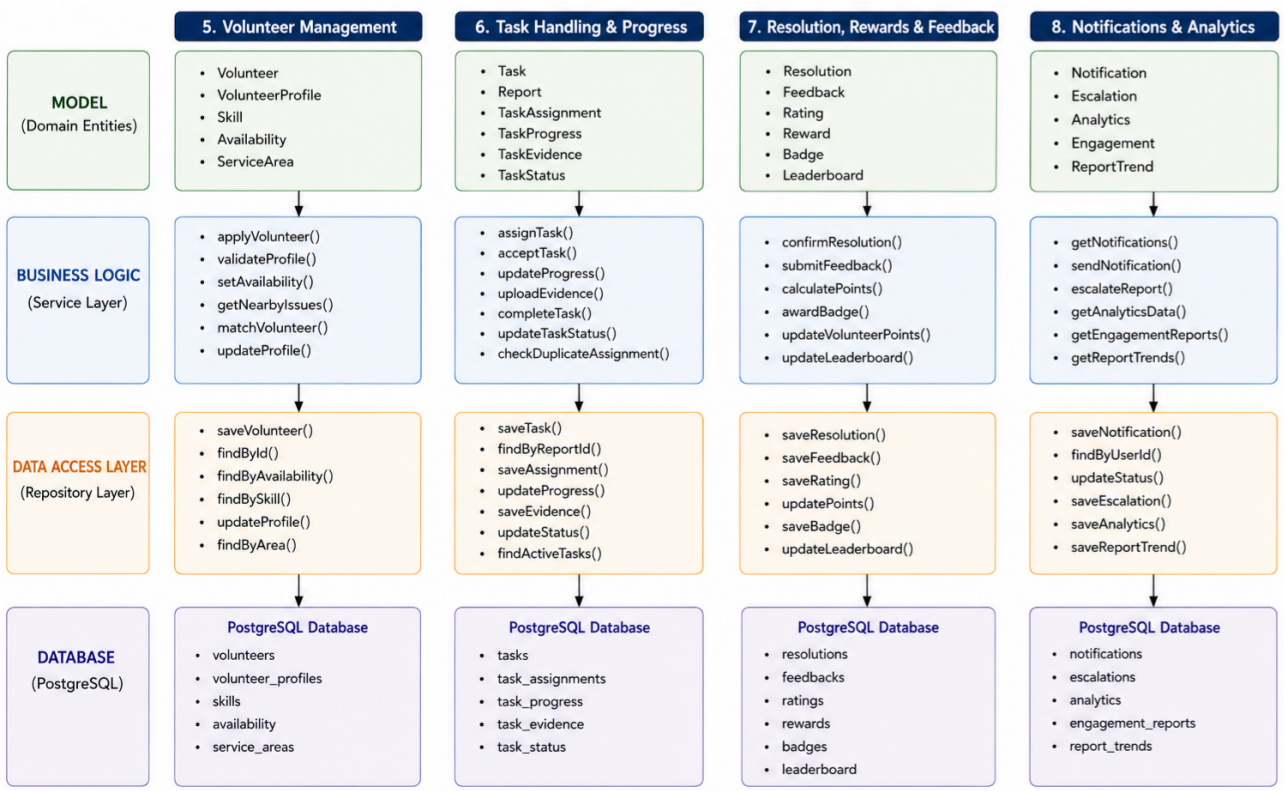


Figure 3: Feature-Wise Three-Tier Architecture – Diagram 2

4.1. Brief Explanation

Figures 2 and 3 together present the complete feature-wise three-tier architecture of AmarShohor. The first diagram covers User Management, Issue Reporting, Community Validation, and Map and Heatmap. The second diagram covers Volunteer Management, Task Handling and Progress, Resolution, Rewards and Feedback, and Notifications, Escalation and Analytics. In every module, the **Model layer** contains the main domain entities. The **Business Logic layer** performs validation, decision-making, duplicate detection, community verification, task matching, progress handling, resolution confirmation, reward calculation, notification generation, and analytics processing. The **Data Access layer** contains repository operations used to save, retrieve, update, and delete information. Finally, the **PostgreSQL database** stores all persistent data. This separation ensures that changes in one layer have minimal impact on the other layers. It also improves maintainability, testing, scalability, and team collaboration.

5. Software Requirements Specification (SRS)

5.1. Functional Requirements

The system defines 36 functional requirements (FR-1 . . . FR-36), several of which are broken down into multiple user stories. Each is listed below with its priority and current status, the requirement statement, its user story, and its acceptance criteria.

FR-1 — Phone Registration

Requirement: The system shall allow users to register using their own phone number and email.

US-1 – Phone Registration

Priority: High • *Status:* To Do

User story: As a User, I want to register with my phone number and verify OTP, so that my account is secure.

Acceptance criteria:

- OTP is sent and must be confirmed before account activation.
- Passwords are stored using hashing, never in plain text.
- Duplicate registration with the same verified number is rejected.

FR-2 — Login

Requirement: The system shall allow users to log in with name and phone number or email.

US-2 – Login

Priority: High • *Status:* To Do

User story: As a User, I want to log in with my credentials, so that I have access to my account.

Acceptance criteria:

- Login succeeds only with correct credentials.
- Login isn't completed with incorrect credentials.

FR-3 — Profile Management

Requirement: The system shall allow users to update their name, phone number, email, photos.

US-3 – Profile Management

Priority: Medium • *Status:* To Do

User story: As a User, I want to update my profile, so that my information stays up to date.

Acceptance criteria:

- Name, photo, and contact details can be edited.
- Changes are saved immediately and reflected across the app.
- Invalid inputs (e.g., malformed phone number) are rejected with an error message.

FR-4 — Password Recovery

Requirement: The system shall allow users to recover a forgotten password using their phone number or email.

US-4 – Password Recovery

Priority: High • *Status:* To Do

User story: As a User, I want to recover my password via OTP or email link, so that I regain access if I

forget it.

Acceptance criteria:

- Password is recovered with OTP.

FR-5 — Volunteer Application

Requirement: The system shall let a Citizen apply to become a Volunteer with skills, availability, and zones.

US-5 – Volunteer Application

Priority: Medium • *Status:* To Do

User story: As a Citizen, I want to apply for Volunteer status with my skills, zones, and availability, so that I can be matched to relevant community work.

Acceptance criteria:

- The application captures skills, preferred zones, and availability windows.
- The applicant can view and edit the application before finalizing.
- Applying does not change existing permissions until approved.

FR-6 — Two Language Support

Requirement: The system shall provide the interface in Bangla and English.

US-6 – Two Language Support

Priority: Medium • *Status:* To Do

User story: As a Citizen, I want to choose my preferred language (Bangla/English), so that I can use the system comfortably.

Acceptance criteria:

- Bangla and English both are supported.
- The selected language persists across sessions.
- All primary UI labels, buttons, and notifications are translated.

FR-7 — Role Based Access

Requirement: The system shall restrict features by role (Citizen, Volunteer, Visitor).

US-7 – Role Based Access

Priority: High • *Status:* To Do

User story: As a Citizen, I want features restricted by my role, so that I only see tools meant for me and others can't misuse restricted features.

Acceptance criteria:

- Each role has a distinct, documented permission set.
- Accessing a restricted feature returns an access-denied response.

- Role changes take effect without requiring re-login.

FR-8 — Report Submission with Attachments

Requirement: The system shall allow report submission with photo, video, or file attachments.

US-8.1 – Report Creation with Photo

Priority: High • *Status:* To Do

User story: As a Citizen, I want to attach a photo to my report, so that I can provide visual proof.

Acceptance criteria:

- A photo can be captured directly or chosen from the device gallery.
- At least one photo can be attached, and supported image formats are accepted.
- Oversized or unsupported files are rejected with a clear message.
- The attached photo is visible as a preview before submission.

US-8.2 – Report Creation with Short Video

Priority: High • *Status:* To Do

User story: As a Citizen, I want to attach a short video, so that I can show issues that a photo can't fully capture.

Acceptance criteria:

- A short video can be recorded directly or chosen from the device gallery.
- Video length is limited to a set maximum duration, and longer clips are rejected or trimmed.
- Oversized or unsupported video files are rejected with a clear message.
- The attached video can be previewed before submission.

US-8.3 – Report Submission within a Minute

Priority: Medium • *Status:* To Do

User story: As a Citizen, I want to submit a report in under a minute, so that reporting doesn't feel like a chore.

Acceptance criteria:

- The report form fits on a single screen with only essential fields required.
- Location and category are pre-filled automatically, with the option to adjust.
- A complete report with one attachment can be submitted in under a minute under normal conditions.
- On submission, the report is created with status "Pending" and appears in the feed.

FR-9 — Categorised Issue Reporting

Requirement: The system shall let Citizens assign a category to each report.

US-9.1 – Categorised Issue Reporting

Priority: High • *Status:* To Do

User story: As a Citizen, I want to pick a category when I report an issue, so that my report reaches the right department and is easy to find.

Acceptance criteria:

- A predefined list of categories (e.g., road, water, waste, electricity, safety) is displayed during report creation.
- Citizen must select exactly one category before submitting a report.
- An "Other" category option is available for issues that do not match the predefined categories.
- The selected category is associated with the report and used for routing, filtering, and map-based display.
- Citizen can edit the selected category before final report submission.

US-9.2 – New Category Addition

Priority: High • *Status:* To Do

User story: As a Citizen, I want to suggest a new issue type when none of the listed categories fit, so that uncommon problems can still be reported accurately instead of being forced into "Other".

Acceptance criteria:

- When choosing "Other," the Citizen can enter a short custom label describing the issue type.
- The report is submitted immediately using the custom label so reporting is never blocked.
- The suggested type is recorded and grouped so that repeated suggestions can be promoted into an official category.
- Once a suggested type is adopted as an official category, existing reports using that label are linked to it.

FR-10 — Duplicate Report Detection

Requirement: The system shall check whether a new report matches an existing unresolved report within the area.

US-10 – Duplicate Report Detection

Priority: High • *Status:* To Do

User story: As a Citizen, I want to be told when I'm about to report an issue that already exists nearby, so that I can support the existing report instead of creating a duplicate.

Acceptance criteria:

- Unresolved issues of the same category within a set radius (e.g., 100m) are checked.
- Citizen is prompted "This issue already exists. Support it instead?"
- Duplicate-linked reports increment the original's supporter count.

FR-11 — Report Location Management

Requirement: The system shall allow citizens to provide and update the location information of a submitted report.

US-11.1 – Add Report Location

Priority: Medium • *Status:* To Do

User story: As a Citizen, I want to provide the location of an issue, so that the responsible volunteer can identify the problem area.

Acceptance criteria:

- Citizen can select a location while creating a report.
- Location information is stored with the submitted report.
- Citizen can view the selected location before final submission.

US-11.2 – Update Report Location

Priority: High • *Status:* To Do

User story: As a Citizen, I want to modify the report location before submission, so that incorrect location information can be corrected.

Acceptance criteria:

- Citizen can edit location details before submission.
- Updated location information is saved with the report.
- Citizen can view the latest location associated with the report.

FR-12 — Report Status Tracking

Requirement: The system shall allow citizens to view the current status of submitted reports.

US-12.1 – Track Report Progress

Priority: High • *Status:* To Do

User story: As a Citizen, I want to track my report status, so that I know the progress of my submitted issue.

Acceptance criteria:

- Citizen can view the current status of submitted reports.
- Report status is updated when progress changes.
- Status history is maintained for each report.

US-12.2 – Receive Status Updates

Priority: Medium • *Status:* To Do

User story: As a Citizen, I want to receive updates about my report status, so that I know when actions are taken.

Acceptance criteria:

- Citizen receives a notification after report status changes.
- Notifications are associated with the related report.
- Citizen can view previous report status notifications.

FR-13 — View Previous Reports
Requirement: The system shall allow users to view their previously submitted reports.

US-13.1 – View Submitted Reports
Priority: High • *Status:* To Do
User story: As a Citizen, I want to view my previous reports, so that I can review my complaint history.
Acceptance criteria:

- Citizen can view reports submitted by their account.
- Report details include category and status.
- Citizen can select a submitted report to view details.

US-13.2 – Filter Report History
Priority: Low • *Status:* To Do
User story: As a Citizen, I want to filter my reports, so that I can find specific complaints easily.
Acceptance criteria:

- Citizen can filter submitted reports by status.
- Citizen can filter submitted reports by date.
- Reports matching the selected filter are displayed.

FR-14 — Confirm Existing Issue
Requirement: The system shall allow citizens to confirm existing reports.

US-14.1 – Confirm Existing Issue
Priority: Medium • *Status:* To Do
User story: As a Citizen, I want to confirm an existing report, so that similar issues can receive more attention.
Acceptance criteria:

- Citizen can confirm an available report.
- User confirmation is recorded with citizen and report information.
- Report confirmation count is updated after confirmation.

FR-15 — Report Priority
Requirement: The system shall allow authorized users to assign priority levels to reports.

US-15.1 – Assign Priority Level

Priority: High • *Status:* To Do

User story: As a Citizen, I want to set priority levels to reports, so that urgent issues can be handled first.

Acceptance criteria:

- Authorized user can select a priority level for a report.
- Assigned priority level is stored with the report.
- Priority information is displayed with the report.

US-15.2 – View Report Priority

Priority: High • *Status:* To Do

User story: As a Citizen, I want to see the priority level of a report, so that I understand its urgency.

Acceptance criteria:

- Citizen can view the priority level of public reports.
- Updated priority information is displayed with the report.
- Latest priority status is maintained for the report.

FR-16 — Volunteer Task Management

Requirement: The system shall allow volunteers to view and manage assigned tasks related to reports.

US-16.1 – View Available Tasks

Priority: Low • *Status:* To Do

User story: As a Volunteer, I want to view available tasks, so that I can select issues that I can help resolve.

Acceptance criteria:

- Volunteer can view available tasks related to unresolved reports.
- Task details include report category and location.
- Volunteer can select an available task.

US-16.2 – Accept Assigned Task

Priority: Medium • *Status:* To Do

User story: As a Volunteer, I want to accept a task, so that I can start working on resolving an issue.

Acceptance criteria:

- Volunteer can accept an available task.
- Selected task is assigned to the volunteer account.
- Task status is updated after acceptance.

FR-17 — Task Progress Update

Requirement: The system shall allow volunteers to update the progress of assigned tasks.

US-17.1 – Update Task Status

Priority: Medium • *Status:* To Do

User story: As a Volunteer, I want to update task progress, so that citizens can know the current resolution status.

Acceptance criteria:

- Volunteer can change the status of an assigned task.
- Each status update is stored with the related task.
- Latest task status is displayed.

US-17.2 – Submit Task Completion Evidence

Priority: Low • *Status:* To Do

User story: As a Volunteer, I want to submit completion evidence, so that completed work can be verified.

Acceptance criteria:

- Volunteer can upload completion evidence for a task.
- Uploaded evidence is associated with the completed task.
- Authorized user can review submitted completion evidence.

FR-18 — Map-Based Report Display

Requirement: The system shall display submitted reports according to their geographical locations.

US-18.1 – View Reports on Map

Priority: High • *Status:* In Progress

User story: As a Citizen, I want to view reports on a map, so that I can identify issue locations.

Acceptance criteria:

- User can view report locations on a map interface.
- Report information is shown when a map location is selected.
- Map displays updated reports when new reports are added.

US-18.2 – View Area Issues

Priority: Medium • *Status:* To Do

User story: As a Volunteer, I want to view nearby reported issues, so that I can stay informed about problems in my area.

Acceptance criteria:

- Citizen can view reports based on selected locations.
- Report categories are displayed on the map.
- Citizen can view report details from the map.

FR-19 — Emergency Report Marking

Requirement: The system shall allow citizens to mark reports as emergency cases.

US-19.1 – Mark Emergency Report

Priority: High • *Status:* In Progress

User story: As a Citizen, I want to mark an issue as an emergency, so that urgent problems receive faster attention.

Acceptance criteria:

- Citizen can select an emergency option during report submission.
- Emergency status is stored with the report.
- Emergency reports are displayed separately.

US-19.2 – Review Emergency Reports

Priority: High • *Status:* To Do

User story: As a Volunteer, I want to identify emergency reports, so that I can prioritize urgent tasks.

Acceptance criteria:

- Volunteer can identify highlighted emergency reports.
- Volunteer can view emergency report details.
- Emergency status is updated when the issue is resolved.

FR-20 — Notifications

Requirement: The system shall notify users about important activities related to reports and tasks.

US-20.1 – Receive Report Notifications

Priority: High • *Status:* In Progress

User story: As a Citizen, I want to receive notifications about my reports, so that I stay updated on progress.

Acceptance criteria:

- Citizen receives notifications for report status changes.
- Notifications are sent to the related citizen.
- Notification records are stored.

US-20.2 – Receive Task Notifications

Priority: High • *Status:* To Do

User story: As a Volunteer, I want to receive task-related notifications, so that I know about assigned activities.

Acceptance criteria:

- Volunteer receives notifications about assigned tasks.
- Volunteer receives notifications about task updates.

- Volunteer can view task notification history.

FR-21 — Emergency Flagging & Verification

Requirement: The system shall allow reports to be flagged as emergencies.

US-21.1 – Flag Report as Emergency

Priority: High • *Status:* To Do

User story: As a Citizen, I want to flag a report as an emergency, so that urgent issues receive immediate visibility and priority.

Acceptance criteria:

- User can mark a report as "Emergency" during report submission.
- Emergency reports display a distinct visual indicator (e.g., red badge).
- Emergency reports appear higher in relevant lists and maps.
- Emergency status is stored with the report.

US-21.2 – Verify Identity Before Emergency Broadcast

Priority: High • *Status:* To Do

User story: As a Citizen, I want emergency reports to require identity verification before they are broadcast, so that misuse of emergency alerts is prevented.

Acceptance criteria:

- Emergency reports require a verified account before submission.
- Unverified users receive a verification prompt.
- Emergency report cannot be published until verification succeeds.
- Verified emergency reports are broadcast immediately.

FR-22 — Verify Report Evidence

Requirement: The system shall allow volunteers to verify evidence attached to reports.

US-22.1 – Verify Report Evidence

Priority: High • *Status:* To Do

User story: As a Volunteer, I want to review report evidence, so that I can validate the provided information before taking action.

Acceptance criteria:

- Evidence is verified before being relied upon for task decisions.

FR-23 — Receive Nearby Issue Alerts

Requirement: The system shall notify volunteers about nearby issues requiring assistance.

US-23.1 – Receive Nearby Issue Alerts

Priority: High • *Status:* To Do

User story: As a Volunteer, I want to receive alerts about nearby issues, so that I can choose suitable tasks.
Acceptance criteria:

- Volunteers are notified of new issues within their preferred service area.

FR-24 — Comment on a Report
Requirement: The system shall allow users to comment on and share reports.

US-24.1 – Comment on a Report
Priority: Medium • *Status:* To Do
User story: As a User, I want to comment on a report, so that I can discuss the issue with others.
Acceptance criteria:

- User can add comments.
- Comments display author and timestamp.
- Comments appear in chronological order.
- Empty comments cannot be submitted.

FR-25 — Search and Filter Reports
Requirement: The system shall allow users to search and filter reports.

US-25.1 – Search Reports
Priority: Medium • *Status:* To Do
User story: As a User, I want to search reports using keywords, so that I can quickly locate relevant issues.
Acceptance criteria:

- Search supports report title and description.
- Matching reports are displayed.
- No-result message appears when applicable.
- Search updates without reloading the page.

US-25.2 – Filter Reports
Priority: Low • *Status:* To Do
User story: As a User, I want to filter reports by category, status, and date, so that I can narrow down the information I need.
Acceptance criteria:

- User can select multiple filters.
- Filters update results immediately.
- User can clear all filters.

- Selected filters remain active until changed.

FR-26 — Public Analytics Dashboard

Requirement: The system shall provide aggregated trends, engagement statistics, and summary analytics.

US-26.1 – View Public Analytics Dashboard

Priority: High • *Status:* To Do

User story: As a User, I want to view public analytics and trends, so that I can understand community issues and platform activity.

Acceptance criteria:

- Dashboard displays charts and statistics.
- Data updates automatically.
- Users can filter analytics by category or time period.
- Dashboard loads successfully on all supported devices.

FR-27 — Escalate Unresolved Issue

Requirement: The system shall allow unresolved or highly critical reports to be escalated for further attention.

US-27.1 – Escalate Unresolved Issue

Priority: High • *Status:* To Do

User story: As a Citizen, I want unresolved issues to be escalated, so that they receive additional attention.

Acceptance criteria:

- Citizens can request escalation only for unresolved reports.
- The system records the escalation request.
- Escalated reports receive a higher priority label.
- Escalation history is stored.

FR-28 — Impact Info on Reported Issue

Requirement: The system shall allow citizens to provide impact information about reported issues.

US-28.1 – Impact Info on Reported Issue

Priority: High • *Status:* To Do

User story: As a Citizen, I want to describe the impact of an issue, so that volunteers can understand its severity.

Acceptance criteria:

- Citizen can provide impact details during report submission.

- Impact information is stored with the related report.
- Volunteers can view impact information while handling reports.

FR-29 — Add Notes

Requirement: The system shall allow volunteers to add notes while working on assigned tasks.

US-29.1 – Add Notes

Priority: High • *Status:* To Do

User story: As a Volunteer, I want to add notes about a task, so that important information can be recorded during issue resolution.

Acceptance criteria:

- Notes are added to the task record and are visible during subsequent reviews.

FR-30 — Additional Evidence

Requirement: The system shall allow citizens to provide additional evidence to support the validity of submitted reports.

US-30.1 – Additional Evidence

Priority: Medium • *Status:* To Do

User story: As a Citizen, I want to provide additional evidence for my report, so that volunteers can better understand and verify the issue.

Acceptance criteria:

- Citizen can add additional evidence to an existing submitted report.
- Added evidence is linked with the corresponding report.
- Volunteer can access available supporting evidence during report review.

FR-31 — Filter and Sort Reports

Requirement: The system shall allow users to filter and sort reports based on available criteria.

US-31.1 – View Previous Reports Based on Criteria

Priority: Low • *Status:* To Do

User story: As a User, I want to filter previous reports by category, status, and date, so that I can find specific reports easily.

Acceptance criteria:

- Provides filtering options for available report attributes.
- Displays reports matching selected filters.
- Users can clear applied filters.

US-31.2 – Sort Reports

Priority: Medium • *Status:* To Do

User story: As a User, I want to sort reports by priority and date, so that I can view important issues first.
Acceptance criteria:

- Users can sort displayed reports.
- Arranges reports according to selected sorting options.
- Maintains accurate report information during sorting.

FR-32 — Save Important Reports
Requirement: The system shall allow users to save reports for later viewing.

US-32.1 – Save Important Reports
Priority: Medium • *Status:* To Do
User story: As a Citizen, I want to save important reports, so that I can easily access them later.
Acceptance criteria:

- Reports can be bookmarked.
- Stores bookmarked reports under the user account.
- Saved reports can be removed.

FR-33 — Volunteer Profile Management
Requirement: The system shall allow volunteers to update their skill information, service area, and availability status.

US-33.1 – Update Volunteer Info
Priority: Medium • *Status:* To Do
User story: As a Volunteer, I want to update my skills and expertise, so that I can receive tasks suitable for my abilities.
Acceptance criteria:

- Volunteer can add or update skill categories in the volunteer profile.
- Updated skill information is saved successfully.
- Volunteer skills are displayed when suitable tasks are identified.

US-33.2 – Update Volunteer Service Area
Priority: High • *Status:* To Do
User story: As a Volunteer, I want to update my preferred service area, so that I can work on issues within my selected locations.
Acceptance criteria:

- Volunteer can select or modify preferred service areas.
- Updated service area information is stored in the volunteer profile.

- Available tasks can be associated with the selected service areas.

US-33.3 – Update Volunteer Availability
Priority: High • *Status:* To Do
User story: As a Volunteer, I want to update my availability status, so that the platform knows when I can accept tasks.
Acceptance criteria:

- Volunteer can set availability status (available/unavailable).
- Current availability information is displayed in the volunteer profile.
- Availability status is updated after any change.

FR-34 — Volunteer Feedback and Rating
Requirement: The system shall allow citizens to provide feedback and ratings on volunteer task completion.

US-34.1 – Rate Volunteer Services
Priority: Medium • *Status:* To Do
User story: As a Citizen, I want to rate the volunteer service after issue resolution, so that the quality of support can be evaluated.
Acceptance criteria:

- Citizen can provide a rating after task completion.
- Submitted ratings are stored with the completed task.
- Volunteer performance information is updated based on received ratings.

US-34.2 – Provide Volunteer Feedback
Priority: Medium • *Status:* To Do
User story: As a Citizen, I want to provide feedback about volunteer work, so that service quality can be improved.
Acceptance criteria:

- Citizen can submit written feedback for completed tasks.
- Feedback is linked to the related volunteer task.
- Volunteer can view feedback received for completed tasks.

FR-35 — Task Participation Requests
Requirement: The system shall allow volunteers to request participation in available tasks.

US-35.1 – Request Tasks
Priority: High • *Status:* To Do
User story: As a Volunteer, I want to request participation in available tasks, so that I can contribute to

resolving community issues.

Acceptance criteria:

- Volunteer can submit a request to participate in an available task.
- Task participation requests are stored with volunteer and task information.
- Volunteer can view the status of submitted task requests.

US-35.2 – Withdraw Tasks

Priority: Medium • *Status:* To Do

User story: As a Volunteer, I want to withdraw my task request, so that I can manage my commitments.

Acceptance criteria:

- Volunteer can cancel a pending task participation request.
- The system updates the request status after withdrawal.
- Withdrawn requests are removed from active request lists.

FR-36 — Deactivate Account

Requirement: The system shall allow users to deactivate their accounts when they no longer want to use the platform.

US-36.1 – Deactivate Account

Priority: Medium • *Status:* To Do

User story: As a User, I want to deactivate my account, so that I can stop using the platform when needed.

Acceptance criteria:

- User can access the account deactivation option from account settings.
- User confirmation is required before account deactivation.
- Deactivated accounts cannot access restricted platform features.
- User account information remains stored according to system rules after deactivation.

5.2. Non-Functional Requirements

- Evidence uploads (photos, videos, files) shall be validated for size and format, with oversized or unsupported files rejected with a clear message.
- The community feed and map/heatmap views shall reflect newly submitted or updated reports within a few seconds, without requiring a full page reload.
- The interface shall be fully available in both Bangla and English, and the selected language shall persist across sessions.

- All passwords and OTP codes shall be stored securely (hashed or encrypted); sensitive account data shall never be exposed through any public-facing API endpoint.
- Emergency reports shall require identity verification before broadcast, and verified emergency reports shall be prioritized and surfaced to relevant users without delay.
- Duplicate-report detection shall complete within a bounded search radius quickly enough not to noticeably delay report submission.
- The PostgreSQL database shall reliably persist all reports, tasks, evidence, comments, and user history without data loss across sessions and devices.
- The application shall remain responsive and usable on both desktop and mobile browsers, including under typical low-bandwidth network conditions.
- The backend API shall return clear, actionable error messages on invalid input so the frontend can inform the user without crashing.
- Role-based access control shall ensure that Citizens, Volunteers, and Visitors can only reach the features permitted for their role.

6. Conclusion

AmarShohor applies an Object-Oriented design to transform its software requirements into a clear and maintainable system structure. The UML class diagram identifies the main entities, their responsibilities, inheritance, associations, and multiplicities. The feature-wise three-tier architecture separates domain models, business logic, data access operations, and PostgreSQL database storage. Dividing the architecture into two diagrams keeps the overall design complete while maintaining readability. The Software Requirements Specification, covering 36 functional requirements and their associated user stories, acceptance criteria, and non-functional requirements, provides a traceable link between the design and what the system is required to do. This design supports modular development, independent testing, easier maintenance, improved scalability, and future extension of the platform. It also creates a clear foundation for implementing AmarShohor as a transparent and community-driven urban issue management system.